HACKATHON IMPLEMENTATION BLUEPRINT

InstituteIQ

AI Operations Copilot for Training Institutes

Problem Statement PS1 | Open Track | MERN Stack + Agentic AI | 24 Hours

React | Node.js | Express | MongoDB | LLM Tool-Calling | JWT + RBAC

The idea in one line

An institute portal where owners, counselors, trainers and accounts staff simply ask questions in plain language. An AI agent answers using safe, read-only queries, shows exactly which query it ran, and turns every change (like sending reminders) into a preview that a human must approve. A built-in scheduler plans a clash-free timetable and re-plans in seconds when something changes.

1. ASK
Plain-language questions across students, fees, EMIs, attendance and trainers. No SQL or filters needed.

2. VERIFY + APPROVE
Query Inspector shows the exact query. Every change is a preview that needs human approval.

3. PLAN
Clash-free weekly timetable that re-plans in seconds and explains every change it made.

Tech stack

Layer	Choice	Why
Frontend	React + Vite + Tailwind CSS	Fast to build, clean modern UI
Backend	Node.js + Express	Simple REST API and the agent loop
Database	MongoDB + Mongoose	Flexible schema, quick to seed
AI agent	LLM with tool-calling (Claude or Gemini API)	Plans, calls tools, loops until done
Auth	JWT + role middleware	Role-based access enforced on the server
Scheduler	Custom constraint solver in JavaScript	No heavy infra, runs in milliseconds
Messaging	WhatsApp click-to-chat link / mock send	Real WhatsApp API not needed for demo

Inside this document

Page 2 Requirements and scope
Page 3 System architecture
Page 4 Portal flow and role access
Page 5 Agentic copilot flow
Page 6 Worked example with real data
Page 7 UI: Copilot screen
Page 8 UI: Dashboard and timetable
Page 9 Data model, tools, APIs | Page 10 24-hour plan

What PS1 Asks vs What We Build

Every requirement from the problem statement mapped to a concrete feature

#	Requirement in PS1	Our feature	Scope
1	Manage students, batches, courses, trainers	CRUD screens with search and filters	MUST
2	Fees and EMIs	EMI schedule per student, status PENDING / PAID / OVERDUE	MUST
3	Hybrid attendance (classroom + online)	Attendance records with mode CLASSROOM or ONLINE and join time	MUST
4	Class recordings	Recording link attached to each session	SHOULD
5	Role-based access (4 roles)	Owner, Counselor, Trainer, Accounts - enforced in backend	MUST
6	Ask questions in plain language	Agent with whitelisted read-only tools (tool-calling)	MUST
7	Show the query it ran	Query Inspector panel under every answer	MUST
8	Changes shown as preview, human approves	pending_actions queue with Approve / Edit / Reject	MUST
9	Every action logged	audit_logs collection and Audit Log screen	MUST
10	Trainer must never see fee details	Role-filtered tools + field masking before data reaches the LLM	MUST
11	Clash-free weekly timetable	Constraint scheduler: trainer, room, batch time, online seats	MUST
12	Re-plan in seconds and explain changes	Trainer unavailable / new batch button + change log	MUST
13	Reconcile UPI / bank payments	Upload CSV, match against expected EMIs	BONUS
14	Completion certificates	Generate PDF certificate per student	BONUS
15	Dropout risk with reasons	Rule-based score + LLM writes the reasons	BONUS

Scope strategy for 24 hours

BUILD FIRST (MUST)
Data + roles, agent with 6 read tools, Query Inspector, role guard, approval queue, audit log, timetable + re-plan.

IF TIME (SHOULD)
Recording links, attendance marking UI, dashboard KPIs and charts, polished empty / error states.

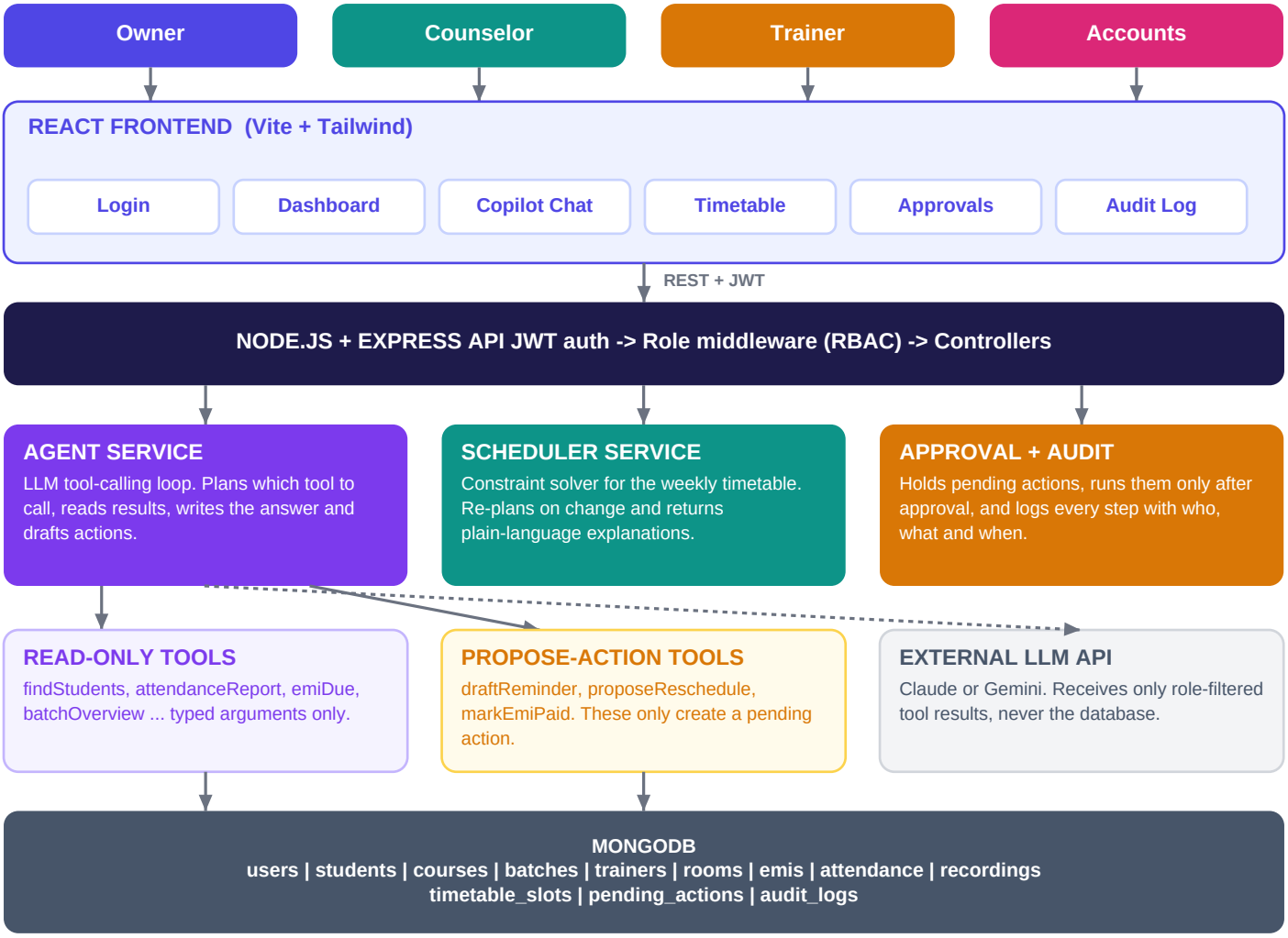
ONLY IF AHEAD (BONUS)
Dropout-risk flags with reasons first (cheap), then certificates, then payment reconciliation.

What judges will look for

- Is it truly agentic? The AI plans, calls tools in a loop and decides next steps - not a single prompt.
- Is it safe? Role checks live in the server, the AI cannot write directly, every action is logged.
- Is it verifiable? Users can see the exact query behind every answer.
- Does the live demo work end to end with realistic data in under 3 minutes?
- Is the timetable truly clash-free, and does the re-plan explain itself?

System Architecture

How React, Node, the AI agent and MongoDB fit together



Five golden rules (this is what makes it safe and impressive)

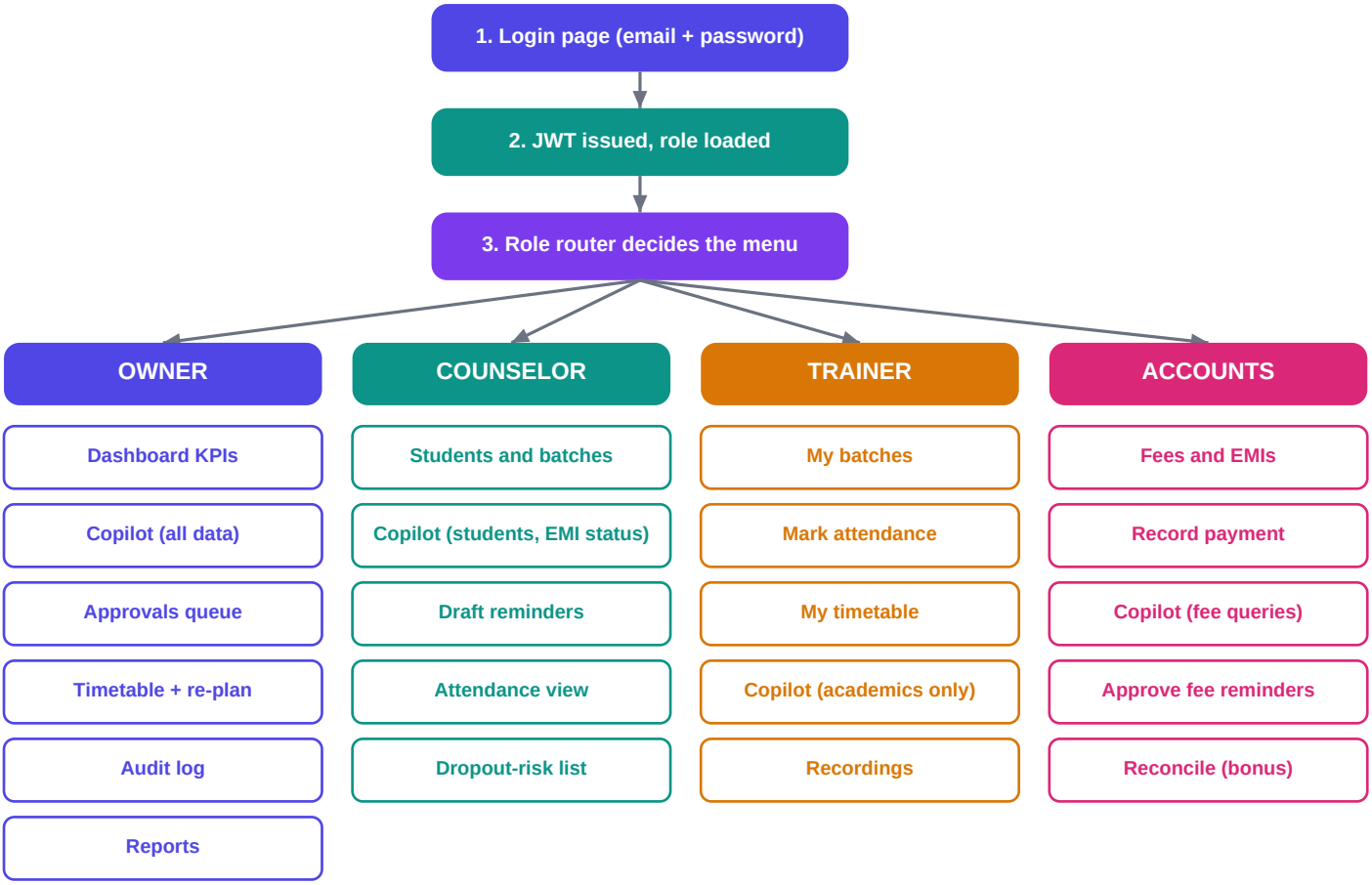
- The LLM never touches the database. It can only call whitelisted tools with typed arguments such as {batch: 'Java', minAbsent: 3}.
- Every tool re-checks the caller's role on the server. A role rule written only in the prompt is not enough.
- Field masking: fee and EMI fields are stripped for trainers BEFORE results go back to the LLM, so it cannot leak what it never saw.
- Write actions never execute directly. They create a pending_action with a preview; a human approves it first.
- Every tool call stores its exact query, which powers the Query Inspector and the audit log.

Why this design wins

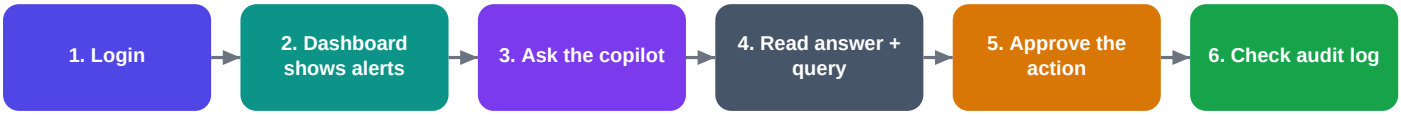
Most teams will build a chatbot that calls the LLM and prints text. InstituteIQ shows a real agent loop, server-side security, a human-in-the-loop approval flow and a verifiable audit trail. Those are exactly the points the problem statement stresses: safe queries, previews, logging and role respect.

Portal Flow: From Login to Approved Action

What each role sees and the daily loop of the portal



The daily loop (owner, Monday morning)



Role access matrix

Data / action	Owner	Counselor	Trainer	Accounts
Student profiles	All	All	Own batches	Name + batch only
Attendance	All	All	Own batches (can mark)	No
Fee and EMI amounts	All	Status only	NEVER	All
Timetable	View + re-plan	View	Own timetable	No
Approve actions	All actions	Own reminders	No	Fee reminders
Audit log	All	No	No	Own actions

Rule of thumb: if a role should not see a field, the backend removes it from the tool result. The prompt only repeats the rule; it is never the only protection.

The Agentic Copilot Flow

How one plain-language question becomes a verified answer and an approved action

Agent loop in code (Node.js, simplified)

```
async function runAgent(user, question) {
  const tools = toolsFor(user.role);           // role-filtered list
  const msgs = [system(user.role), userMsg(question)];
  for (let i = 0; i < 5; i++) {
    const res = await llm.chat(msgs, tools);
    if (!res.toolCalls) return saveAndReturn(res, log);
    for (const call of res.toolCalls) {
      assertAllowed(user.role, call.name);      // server-side guard
      const out = await registry[call.name](call.args, user);
      log.push({ tool: call.name, args: call.args, query: out.query });
      msgs.push(toolResult(call.id, mask(out.data, user.role)));
    }
  }
}
```

Worked Example with Realistic Data

The exact demo question from the problem statement, step by step

Owner: Which students in the Java batch have missed three or more classes and have an EMI due this week? Draft a WhatsApp reminder for each.

A. What the agent does (3 tool calls)

Step	Tool	Arguments	Returns
1	findBatch	{ name: 'Java' }	B101 Java Full Stack, Mon-Fri 6 PM, trainer Amit
2	attendanceReport	{ batch: 'B101', minAbsent: 3 }	4 students with 3 or more absences
3	emiDue	{ students: [4 ids], withinDays: 7 }	3 of them have an EMI due by 23 Oct

B. Query Inspector shows this (read-only)

```
db.attendance.aggregate([
  { $match: { batchId: 'B101', status: 'ABSENT' } },
  { $group: { _id: '$studentId', absent: { $sum: 1 } } },
  { $match: { absent: { $gte: 3 } } } ])
db.emis.find({ studentId: { $in: [...] }, status: 'PENDING', dueDate: { $lte: '2026-10-23' } })
```

C. Answer returned to the user

Student	Classes missed	EMI due on	Amount
Rahul Patil	4	18 Oct 2026	Rs. 4,500
Sneha Kulkarni	3	20 Oct 2026	Rs. 4,500
Omkar Jadhav	5	21 Oct 2026	Rs. 6,000

D. Drafted message and action preview

Hi Rahul, this is a reminder from Demo Institute. Your EMI of Rs. 4,500 is due on 18 Oct. We also noticed you missed 4 classes in the Java batch. Reply if you need help catching up.

ACTION PREVIEW (PENDING)

Send 3 WhatsApp reminders

Approve | Edit | Reject

E. The safety demo: a trainer asks about fees

Trainer: How much fees has Rahul paid so far?

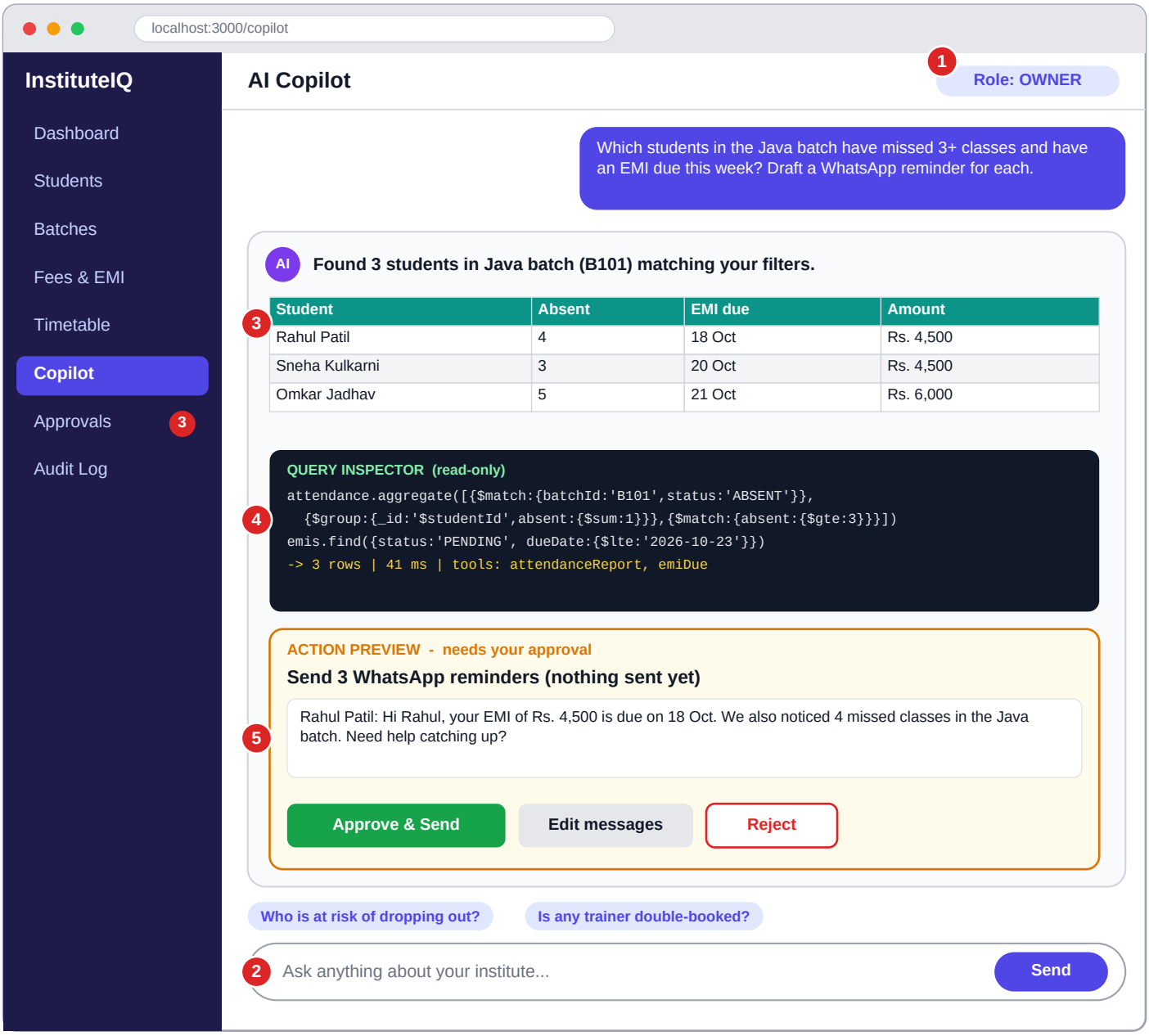
Copilot: I can't access fee details with the Trainer role. I can show Rahul's attendance and assignment progress instead.

audit_logs entry

```
role: trainer | event: DENIED
tool: feeSummary (not in trainer toolset)
at: 2026-10-16 10:42 | by: trainer_07
```

UI Mockup 1: The Copilot Screen

Small wireframe of the most important screen in the demo



What each numbered part does

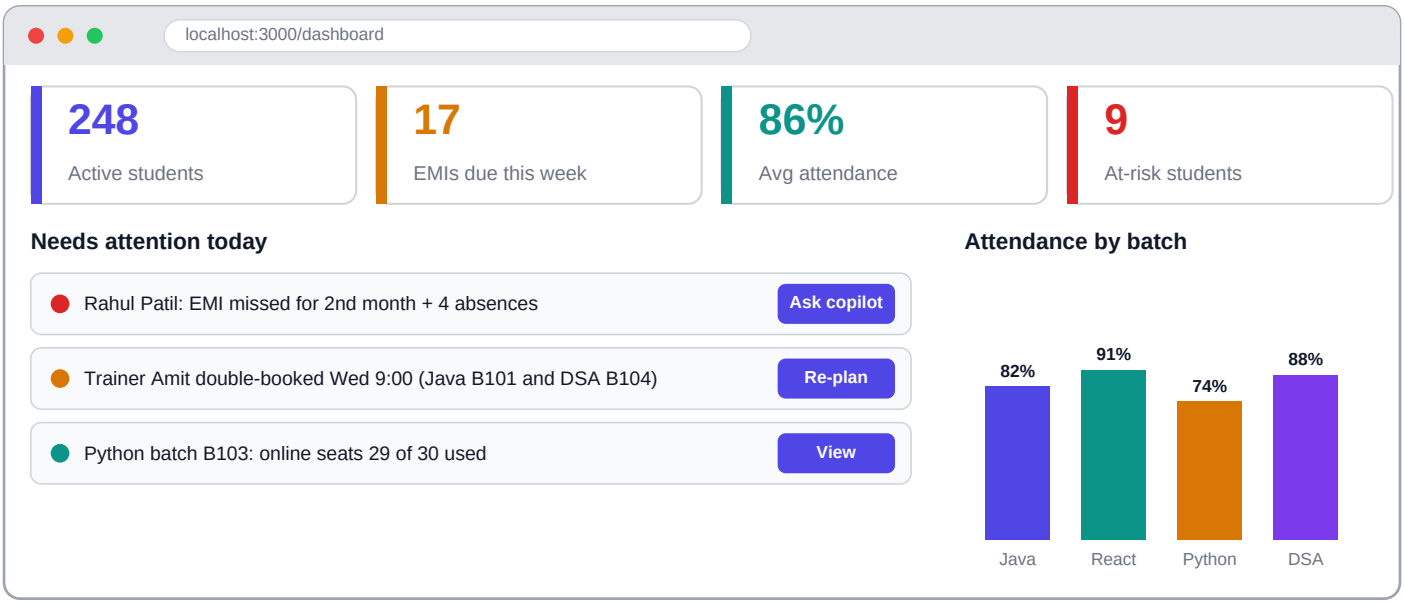
- 1 Role badge: always visible. It decides which tools the AI is allowed to use for this user.
- 2 Chat box + suggestion chips: one place to ask anything. Chips make the live demo smooth.
- 3 Result table: a structured answer instead of a wall of text. Easy to scan and export.
- 4 Query Inspector: exact query, time taken and tools used. Directly satisfies 'show the query it ran'.
- 5 Action preview: nothing is sent until a human clicks Approve. Edit and Reject are logged too.

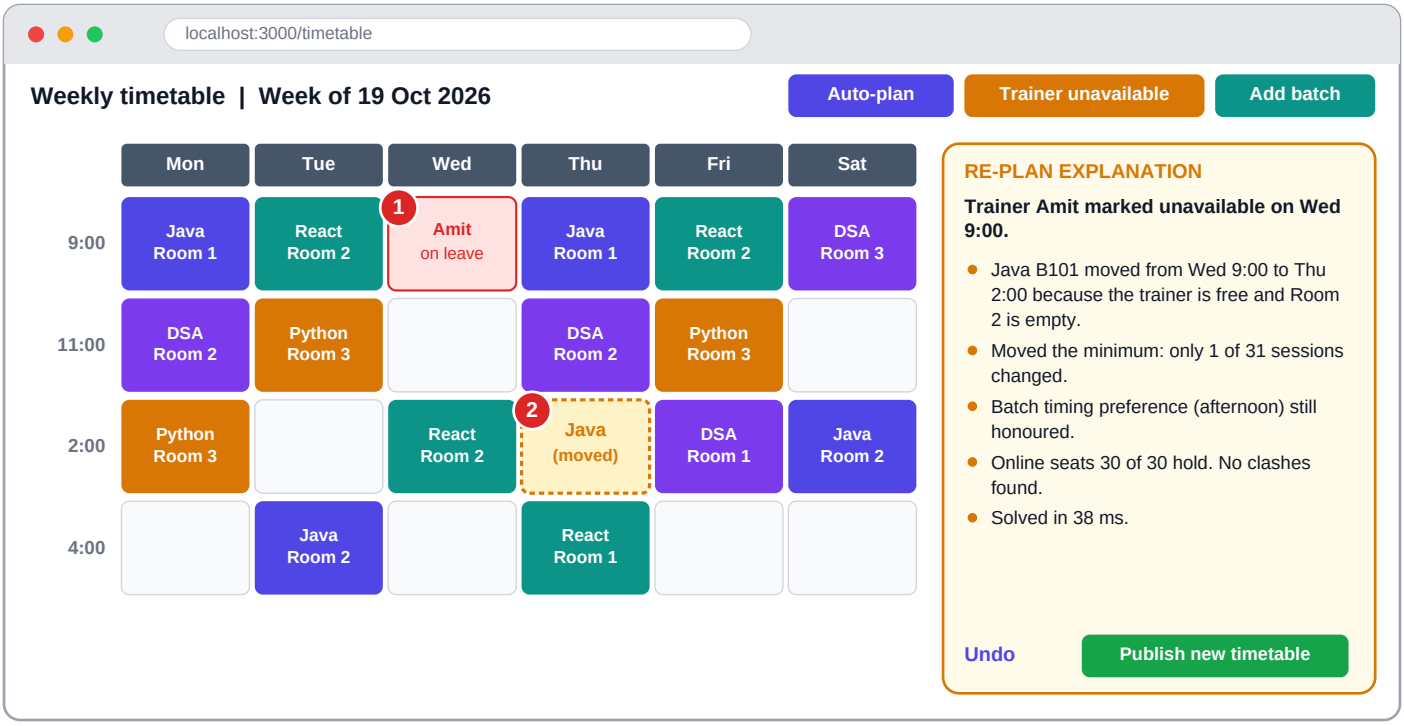
React building blocks and message shape

```
<ChatWindow>  <RoleBadge/>  <MessageBubble/>  <ResultTable/>  <QueryInspector/>  <ActionPreviewCard/>
message = { role: 'assistant', text, table: [...], queries: [{ tool, args, mongo, ms }],
            pendingActionId: 'a_9f2c' }   // Approve -> POST /api/actions/:id/approve
```

UI Mockup 2: Dashboard and Smart Timetable

The dashboard surfaces problems; the timetable re-plans and explains itself





How the scheduler works (simple and fast)

- Inputs: trainer availability, room capacity, batch days and timing, online seat limit.
- Hard rules: no trainer, room or batch in two places at once; room capacity >= batch size.
- Plan: sort batches by how constrained they are, then place each in the first valid slot; backtrack on conflict.
- Re-plan: freeze everything unaffected, move only the broken sessions: same day, then other day, then a substitute trainer with the skill.
- Explain: each move stores a reason string; the LLM only rewrites these into friendly text.

```
function replan(slots, event) {
  const broken = slots.filter(s => clashes(s, event));
  for (const s of broken) {
    const options = candidates(s)   // same day, other day, sub
      .filter(o => isFree(o, slots));
    const best = options.sort(byLeastChange)[0];
    moves.push({ from: s, to: best, why: reason(s, best) });
  }
  return { moves, ms: elapsed() };
}
```

Data Model, Agent Tools and APIs

The technical backbone: what to store, what the AI can call, what the frontend calls

MongoDB collections

Collection	Key fields
users	name, email, passwordHash, role (owner | counselor | trainer | accounts)
students	name, phone, courseId, batchId, status, joinedAt
courses	title, durationWeeks, totalFee
batches	courseId, trainerId, mode (classroom | hybrid), days[], startTime, roomId, onlineSeats
trainers	userId, skills[], availability [{ day, from, to }]
rooms	name, capacity
emis	studentId, installmentNo, amount, dueDate, status (PENDING | PAID | OVERDUE)
attendance	studentId, batchId, date, mode (CLASSROOM | ONLINE), status, joinTime
recordings	batchId, date, url
timetable_slots	batchId, trainerId, roomId, day, start, end, version
pending_actions	type, payload, previewText, createdBy, status (PENDING | APPROVED | REJECTED)
audit_logs	userId, role, event, toolCalls[], query, result, at

Agent tools (what the LLM is allowed to call)

Tool	Type	Allowed roles	Purpose
findStudents	read	owner, counselor, trainer (own batches)	Search students by name, batch, status
attendanceReport	read	owner, counselor, trainer (own batches)	Absences per student, date range
emiDue	read	owner, accounts, counselor (status only)	EMIs due or overdue in a window
feeSummary	read	owner, accounts	Paid vs pending amounts per student
batchOverview	read	all roles	Batch, trainer, room, seats used
trainerSchedule	read	owner, counselor, trainer (own)	Weekly sessions and free slots
dropoutRisk	read	owner, counselor	Score from attendance + payment + assignments
draftReminder	propose	owner, counselor, accounts	Creates pending WhatsApp messages
proposeReschedule	propose	owner	Creates pending timetable change
markEmiPaid	propose	owner, accounts	Creates pending payment update

REST endpoints used by the React app

Method	Endpoint	Purpose
POST	/api/auth/login	Returns JWT with role
GET	/api/students | /api/batches | /api/emis	Normal list screens (role-filtered)
POST	/api/copilot/ask	Runs the agent loop, returns answer + queries + pendingActionId
GET	/api/actions?status=PENDING	Approvals queue
POST	/api/actions/:id/approve | /reject	Human decision, then executor + audit log
POST	/api/timetable/generate | /replan	Auto-plan and re-plan with explanation
GET	/api/audit	Audit log viewer (owner)

Seed data tip: write one script that creates 100+ students, 6 batches, 4 trainers, EMIs and 60 days of attendance, with a few deliberate problems (missed EMIs, a double-booked trainer). Your demo depends on it.

24-Hour Plan, Team Split and Demo Script

Time-boxed so the core is finished well before the deadline

Hours	What to finish	Who
0 - 2	Repo, Vite + Express, MongoDB Atlas, API keys, folder structure, git branches	All
2 - 6	Models + seed script, JWT login + role middleware, login page and app layout	B, D, A
6 - 12	Agent loop + 6 read tools + Query Inspector data; chat UI and result table	C, A
12 - 16	Role guard + field masking tests (trainer vs fees), pending_actions, approve / reject, Approvals UI	C, B, A
16 - 19	Audit log + viewer, dashboard KPIs and alerts	B, A
19 - 22	Timetable generator, re-plan + explanation, timetable UI	D, A
22 - 23.5	Bonus: dropout-risk flags with reasons. Bug fixing, empty states, loading states	All
23.5 - 24	Code freeze, README, rehearse the demo twice	All

Suggested team split (4 members, adjust as needed)

A Frontend
React screens, chat UI, Query Inspector, approval card, timetable grid

B Backend
Express, auth, RBAC middleware, CRUD APIs, audit log

C AI agent
Tool registry, agent loop, prompts, field masking, pending actions

D Scheduler + data
Seed script, constraint solver, re-plan, demo script

3-minute demo script

- 0:00 Show the dashboard: 17 EMIs due, 9 at-risk students. Name the pain in one sentence.
- 0:30 Owner asks the Java batch question. Open the Query Inspector and show the exact query.
- 1:15 Click Approve on the 3 WhatsApp reminders. Open the Audit Log entry.
- 1:45 Switch to Trainer login. Ask about fees. Show the polite refusal and the DENIED log.
- 2:15 Mark Amit unavailable. Timetable re-plans in milliseconds with a clear explanation.
- 2:45 Close with the architecture: tools only, role guard, human approval, audit trail.

Risks and fallbacks

- LLM rate limit or Wi-Fi fails: keep a cached response for the 3 demo questions and a screen recording as backup.
- WhatsApp: use a wa.me click-to-chat link or a mock send. The real Business API is not needed to win.
- Do not let scope creep: finish all MUST items before touching any BONUS item.

Submission checklist (from the hackathon terms)

- List every open-source library and public API in the README (Terms, clause 7).
- All work must be created during the hackathon and be your own (clause 6). Do not reuse old projects.
- Code belongs to TechSummitTrainers after submission (clauses 1 and 5). You may describe your role in your resume.
- Have a README with setup steps, demo logins for all four roles and a short architecture diagram.